IY3660 – Applications of Cryptography

Week 3
One-time pad and stream ciphers

Elizabeth Quaglia
Information Security Group

In this lecture

- One-time pad
- Stream ciphers

One-time pad

One-time pad

- The main ideas deployed in the design of classic ciphers (transposition, polyalphabetic substitution) serve as a basis of modern **symmetric** encryption ciphers
- A natural generalisation of polyalphabetic substitution ciphers led to the idea of **one-time pad** (OTP)
- OTP: Vigenère Cipher with infinite keyword length
    - Let M be a message with n characters
    - Randomly generate a keyword also of length n
    - Encrypt character by character, as with the Vigenère cipher
    - Use keyword only once!
    - Then this is truly an **unbreakable cipher**

One-time pad security

- Let **p** be a plaintext with n characters
- Randomly generate a keyword also of length n
- Encrypt character by character, as with the Vigenère cipher
- Use keyword only once!

- OTP offers perfect security in the **ciphertext-only attack** scenario
  - The ciphertext **c** leaks nothing more about the plaintext **p** than what was known beforehand (except for its length)
  - Formally, this can be expressed in terms of probabilities:

$$\Pr[P = \mathbf{p} \mid C = \mathbf{c}] = \Pr[P = \mathbf{p}]$$

Example: Caesar asks for a big cat to be sent to Rome

l	e	o	p	a	r	d
c	a	b	b	d	f	a
N	E	P	Q	D	W	D

lower case : plaintext
Green: key (a = 0, b= 1, etc)
UPPER CASE: ciphertext

Example: Caesar asks for a big cat to be sent to Rome

l	e	o	p	a	r	d
c	a	b	b	d	f	a
N	E	P	Q	D	W	D

lower case : plaintext
Green: key (a = 0, b= 1, etc)
UPPER CASE: ciphertext

N	E	P	Q	D	W	D
l	x	l	m	k	w	w
c	h	e	e	t	a	h
y	e	c	x	w	s	n
p	a	n	t	h	e	r
k	e	y	q	b	w	s
c	a	r	a	c	a	l

Example: Caesar asks for a big cat to be sent to Rome

l	e	o	p	a	r	d
c	a	b	b	d	f	a
N	E	P	Q	D	W	D

lower case : plaintext
Green: key (a = 0, b= 1, etc)
UPPER CASE: ciphertext

- Every possible 7-letter big cat can arise as a possible plaintext!

N	E	P	Q	D	W	D
l	x	l	m	k	w	w
c	h	e	e	t	a	h
y	e	c	x	w	s	n
p	a	n	t	h	e	r
k	e	y	q	b	w	s
c	a	r	a	c	a	l

Example: Caesar asks for a big cat to be sent to Rome

l	e	o	p	a	r	d
c	a	b	b	d	f	a
N	E	P	Q	D	W	D

lower case : plaintext
Green: key (a = 0, b= 1, etc)
UPPER CASE: ciphertext

N	E	P	Q	D	W	D
l	x	l	m	k	w	w
c	h	e	e	t	a	h
y	e	c	x	w	s	n
p	a	n	t	h	e	r
k	e	y	q	b	w	s
c	a	r	a	c	a	l

- Every possible 7-letter big cat can arise as a possible plaintext!
- If the adversary knew *a priori* (in advance) that a big cat had been asked for, then the ciphertext tells them nothing new

One-time pad on computers

- Represent message as binary data (over alphabet $\{0, 1\}$)
- Operation (XOR): $\mathcal{E}(b, k) = b + k \ mod \ 2$
- Key: stream of random bits of same length of message:

$$0011010110100101101 0 \ldots$$

- Encrypt bit by bit (via XOR)
- Use key only once!

One-time pad: practicalities

- OTP offers perfect security, but…
  - The key K needs to be as long as the message
  - The key K needs to be distributed in advance to the communicating parties
    - Creating a significant *key management* problem
  - Once the key is in place, it enables a single plaintext to be sent securely, at any point in the future
  - **Using the key K more than once is fatal to security!**
    - "Subtracting the ciphertexts" removes the key K, leaving the difference of the plaintexts
    - Recovering the individual plaintexts is then possible
- Yet OTP has been widely used for encryption of diplomatic messages
- It is also the basis of modern **stream ciphers**

Stream ciphers

Stream ciphers

- Stream ciphers attempt to replicate the properties of the one-time pad
- For a one-time pad, we want to produce

$K = K_0, K_1, K_2, \ldots$: a sequence of **random** integers between 0 and 25

(or in the modern setting, between 0 and 1)

but using only a short key – known to be **impossible**!

- Instead, stream ciphers use a short key to produce

$K = K_0, K_1, K_2, \ldots$: a sequence of **pseudorandom** integers between 0 and 25

(in the modern setting, between 0 and 1)

Pseudorandom: indistinguishable from random to a computationally bounded adversary

Stream ciphers

Definition
A keystream generator takes as input a key K (and possibly an initialisation vector IV), performs some initialisation operation, and then generates as output an (infinitely long) stream of bits $K_0, K_1, \ldots$ - called the keystream.

- Bits, not integers, though some stream ciphers produce bytes or words
- IV enables many different key streams to be produced for a single key K
- IV is usually a fixed-width field and typically set to be a counter in applications
- IV is not secret! It is assumed known to users and potential attackers

Stream ciphers

Encryption:

- Decompose the plaintext P into a sequence of bits $P_0, P_1, \ldots$
- XOR the plaintext and keystream bit by bit to create the ciphertext bits $C = C_0, C_1, \ldots$

Decryption:

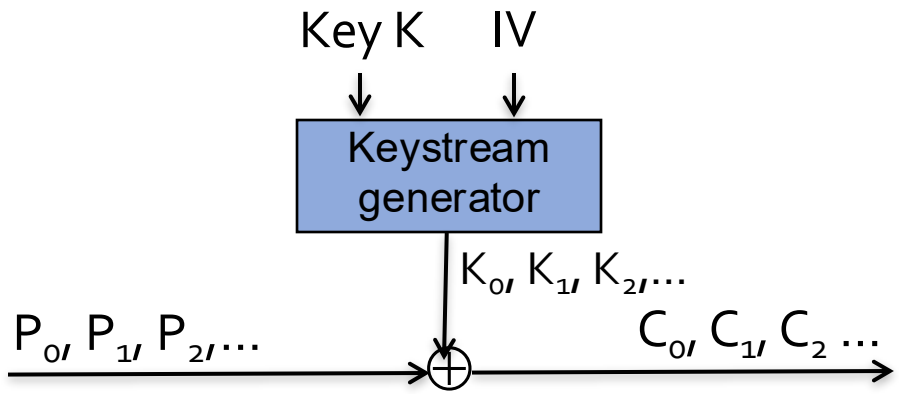
- Decompose the ciphertext C into a sequence of bits $C_0, C_1, \ldots$
- XOR the ciphertext stream and keystream bit by bit to recover the plaintext bits $P = P_0, P_1, P_2, \ldots$

- Importantly, both encryption and decryption are identical
- Decryption undoes encryption because for every i, we have:

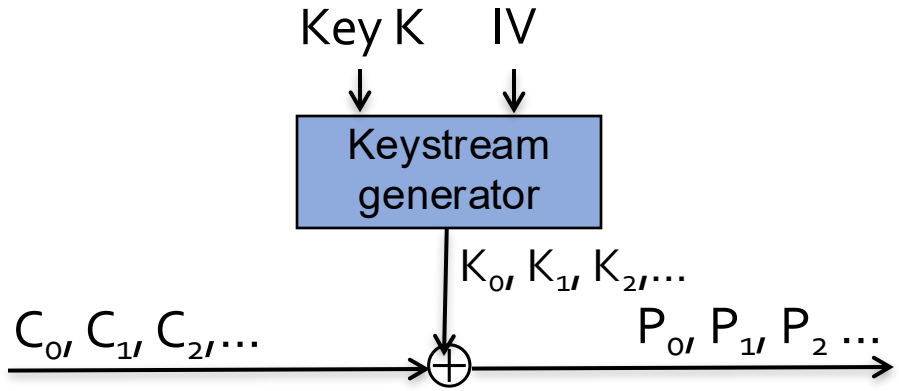
$$(P_i \oplus K_i) \oplus K_i = P_i \oplus (K_i \oplus K_i) = P_i \oplus 0 = P_i$$

Stream ciphers: depiction

Encryption



Decryption



Stream ciphers properties

- Encrypt individual characters of plaintext, one at a time
- Generally fast
- No (or limited) error propagation
- Particularly suitable when no buffering is available and/or characters need to be processed individually (on-the-fly)
- Note that stream ciphers are stateful
  - Parties need to keep in sync for correct decryption
- Approaches to minimise the occurrence of out-of-sync communication
  - Frequent re-initialisation with new IV to resync
  - Use of *self-synchronising* stream ciphers (rarely used, not considered here)

Modern stream ciphers

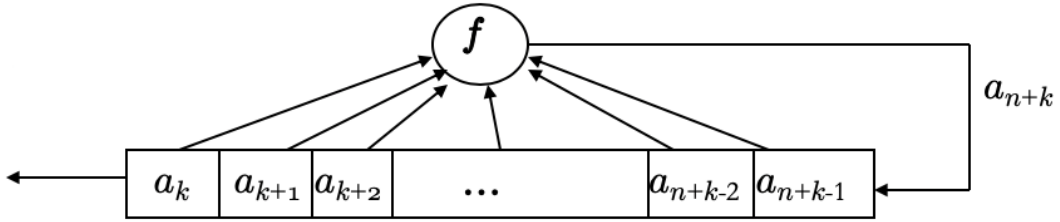
- Stream ciphers standardized by NIST (US) - block ciphers in stream cipher mode:
  - AES and 3DES in counter mode (see later)
- Stream ciphers standardized by ETSI (EU) for mobile phone communications:
  - A5/1-2-3, AES-CTR, SNOW, ZUC
- Dedicated stream ciphers identified by eSTREAM (EU project, 2008):
  - Profile 1: high throughput software applications
    - HC-128, Rabbit, Salsa20/12, SOSEMANUK
  - Profile 2: hardware applications with limited silicon area, power
    - Grain, MICKEY, Trivium
- Other noteworthy stream cipher:
  - ChaCha, a variant of Salsa, now adopted by IETF for use in TLS

Stream ciphers: main constructions

- Based on Linear/Non-Linear Feedback Shift Registers to produce a long pseudorandom keystream (derived from output of the FSRs)
  - Example: Grain, SNOW, ZUC
- Based on a pseudorandom function/permutation **P** and a counter to produce a long pseudorandom keystream (from the output of **P** with counter as input)
  - Examples: ChaCha/Salsa, AES-CTR
- Based on slowly evolving (very) large secret state to produce a long pseudo-random keystream (extracted from the large state)
  - Examples: RC4, HC-128

LFSR-based stream ciphers

- Most popular stream cipher design
- LFSR: linear feedback shift register
  - Uses a n-stage shift register (n bits, or n words)
  - Initial state is stored in the register
  - State (registers) are updated using a linear feedback function



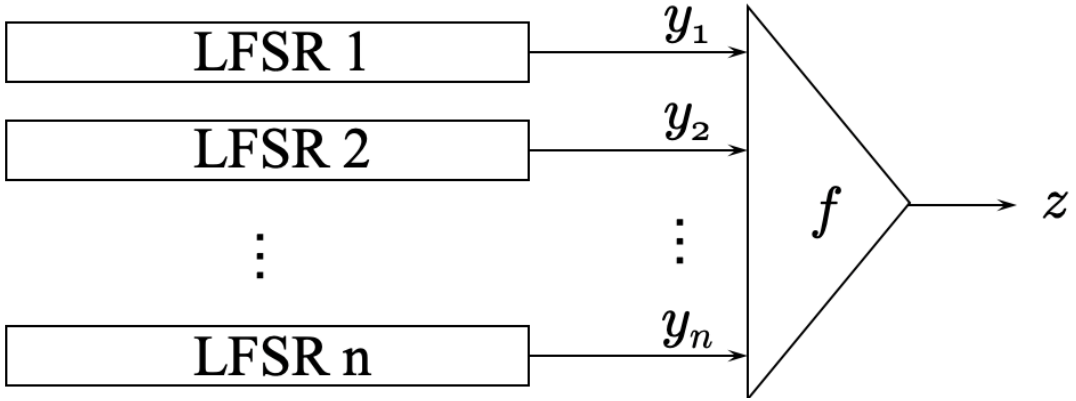
- LFSRs produce long sequences with good statistical properties
  - Could we use them directly as a stream cipher?

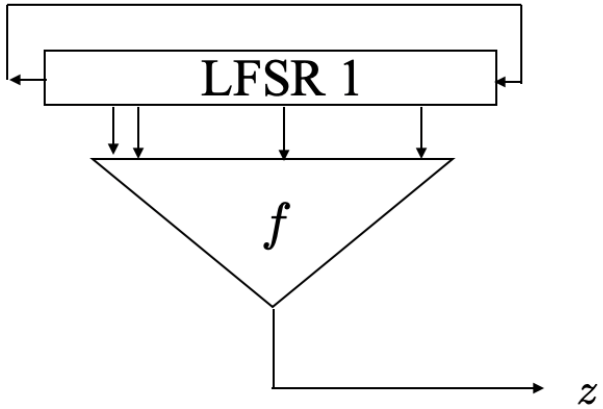
LFSR-based stream ciphers

- Could we use them directly as a stream cipher?
- **No!**
  - The feedback function f is linear
  - There are trivial linear relations between terms of sequence
  - Adversary can use these to predict future terms of the sequence
- To obtain a secure stream cipher we need to combine and/or modify LFSR sequences to destroy their linear properties

LFSR-based stream ciphers

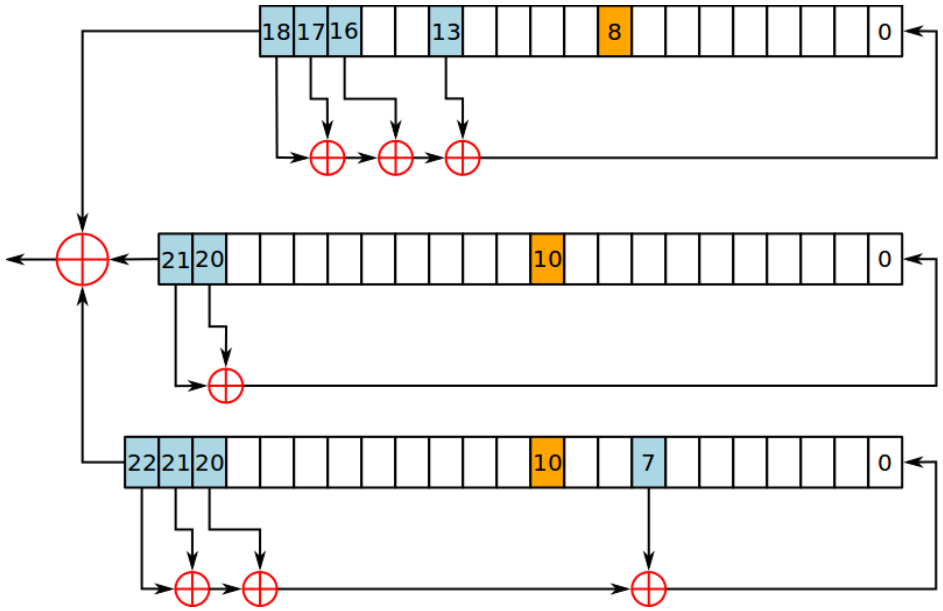
- Classical Constructions:
  - Use of non-linear *combining* function on the output of several LFSRs
  - Use the output of one LFSR to control the clock of other LFSR
  - Combine LFSR and NLFSR





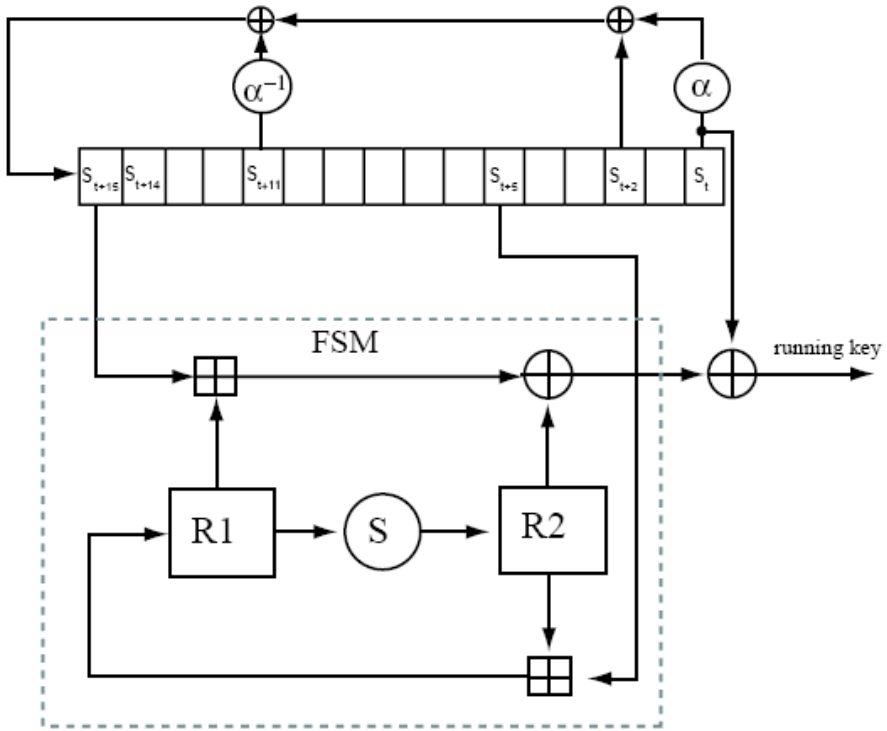
Stream cipher examples: A5/1 (GSM cipher)

- LFSR-based design
- In use since 1980s
- Had to be very fast and have a low gate-count in hardware (114 bits every 4.615ms)
- Subjected to significant amount of cryptanalysis
  - Now considered insecure: attacks recovering key in a few seconds given a few hundred known plaintext bits (achievable in practice)



Stream cipher examples: SNOW 2.0

- Designed at Lund University
- SNOW 2.0
  - ISO Standard
- SNOW 3G
  - UMTS (3G) & LTE (4G)
- Use of block cipher elements
- Very efficient in software



Stream cipher examples: RC4

- Designed by Ron Rivest in 1987
- Originally a trade secret; made public in 1994
- Designed for byte-oriented software
- Elegant design, fast in software, very compact description, easy to implement
- Very popular choice for stream cipher in communication protocols
  - e.g. WEP, WPA, SSL/TLS, MPPE
- Security based on a large secret state S
  - Effective state space is of size $\log_2(2^8!) \sim 1684$ bits
  - State slowly (but continually) evolves in a complex nonlinear way
- Key length is variable

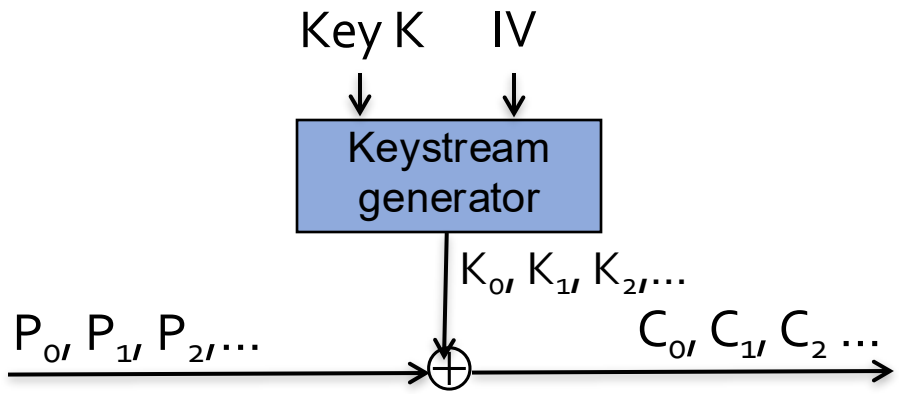
RC4 status

- RC4 *has been* widely adopted in communication protocols:
  - TLS
  - WEP
  - WPA/TKIP
  - Kerberos

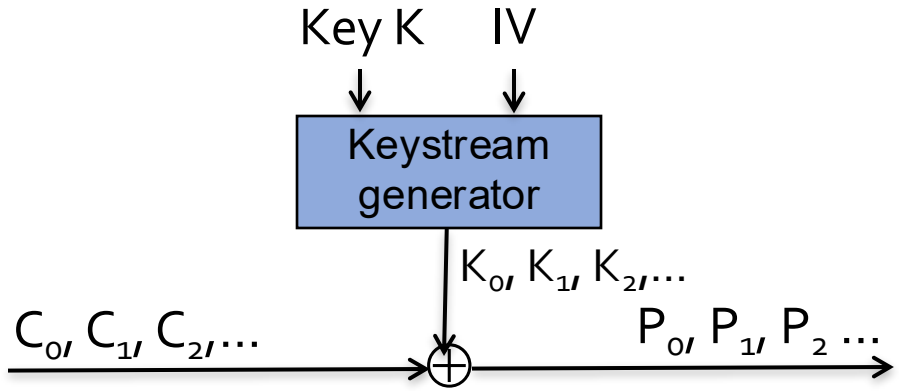
- However it has serious security vulnerabilities and should not be used in any new applications!

Stream ciphers: recall the depiction

Encryption



Decryption



Security issues with stream ciphers: keystream reuse

- Suppose plaintexts P_1, P_2 are encrypted with the same keystream K. So

$$C_1 = P_1 \oplus K \qquad \text{and} \qquad C_2 = P_2 \oplus K$$

- Then $C_1 \oplus C_2 = (P_1 \oplus K) \oplus (P_2 \oplus K) = \ldots = P_1 \oplus P_2$
- Adversary can learn XOR of the plaintexts from XOR of the ciphertexts!
- From $P_1 \oplus P_2$, it *may* be possible to learn the individual plaintexts P_1, P_2.
- This depends heavily on the statistics of the plaintext
  - certainly possible for natural language, application protocols such as HTTP, etc

Security issues with stream ciphers

How does keystream reuse arise in practice?

Example 1: WEP (as used in WLANs) uses the RC4 stream cipher with a 40-bit key K and a 24-bit IV.

- The key is often set by the user once and left fixed forever
- The IV is usually a counter, and is incremented for each frame sent on the network
- Hence, every 2^{24} frames, the key and IV used by RC4's keystream generator will repeat
- This results in repeated keystreams

Security issues with stream ciphers

How does keystream reuse arise in practice?

Example 2: The **Lorenz cipher** was used by German high command in WWII for wireless communications on dozens of communications links.

- Lorenz was a stream cipher operating on 5-bit teletype characters
- August 1941: Operator error led to repeated keystreams – revealed by repeated message indicator "HQIBPEXEZMUG".
- British intercepted two ciphertexts , recovered $P_1 \oplus P_2$, then P_1 and P_2, then the keystream
- From the keystream, the British managed to reconstruct the entire Lorenz cipher…
- … then went on to develop some of the first electronic computers to industrialise the breaking of Lorenz
- … and win the war, without ever seeing an actual Lorenz machine.

Security issues with stream ciphers: statistical weaknesses in keystreams

Keystream biases:

- The keystreams produced by a stream cipher should appear to be random.
- For some stream ciphers, this is not the case.

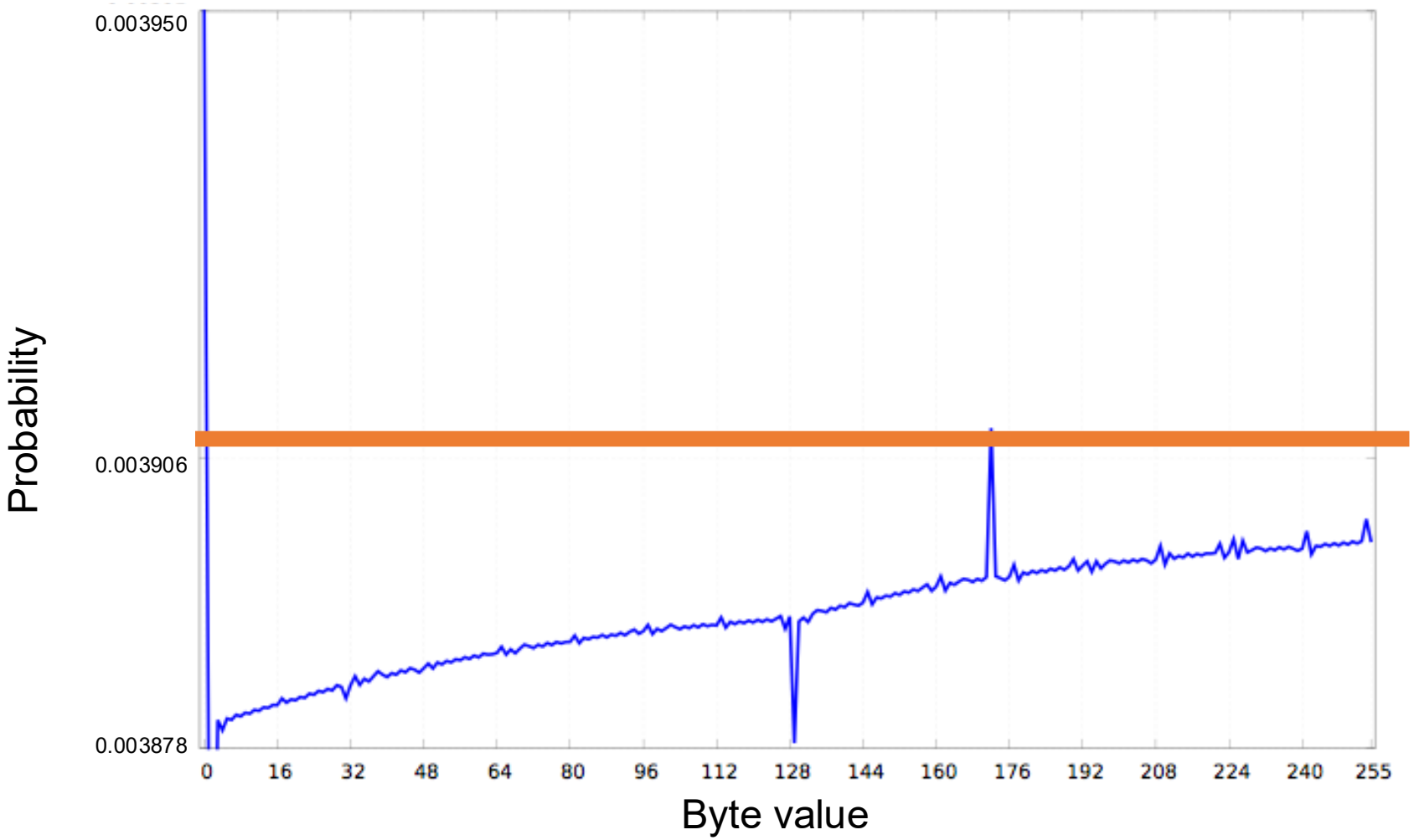
Example: RC4

- RC4 is byte oriented, so would hope that

$$\Pr(K_i = 0x00) = \Pr(K_i = 0x01) = \ldots = \Pr(K_i = 0xff) = 1/256.$$

- But it has been shown that all first 256 keystream bytes of RC4 are biased!
- E.g., Mantin-Shamir showed in 2001 that $\Pr(K_2 = 0x00) = 1/128$.
- These biases can be exploited to produce near-practical attacks on RC4 in applications

RC4 keystream distribution at position 2



1/256 = 0.00390625

(orange line not to scale)

Security issues with stream ciphers: lack of integrity

- Stream ciphers provide no **integrity** against active attackers.
- Recall definition of integrity from last week:

The assurance that data has not been altered in an unauthorised (…) manner since the time that the data was last created, transmitted or stored by an authorised entity.

Lack of integrity in stream ciphers

- Flipping a bit (changing $0 \rightarrow 1$ or $1 \rightarrow 0$) in the ciphertext stream has the effect of producing a bit-flip in the plaintext stream in the same position: $C_i = P_i \oplus K_i$ so $C_i \oplus 1 = P_i \oplus 1 \oplus K_i$
- Hence decryption algorithm will output $P_i \oplus 1$ instead of P_i
- Allows undetectable modification to the plaintext by active adversary
- In fact, others forms of encryption suffer from similar issues
  - Highlights the general fact that encryption does **not** provide integrity
- Can lead to practical attacks on real systems (more later)

Homework

- Read Chapters 1, 3, 4.1, and 4.2 of *Everyday Cryptography*
- Read Chapters 1 and 5 of *Serious Cryptography*